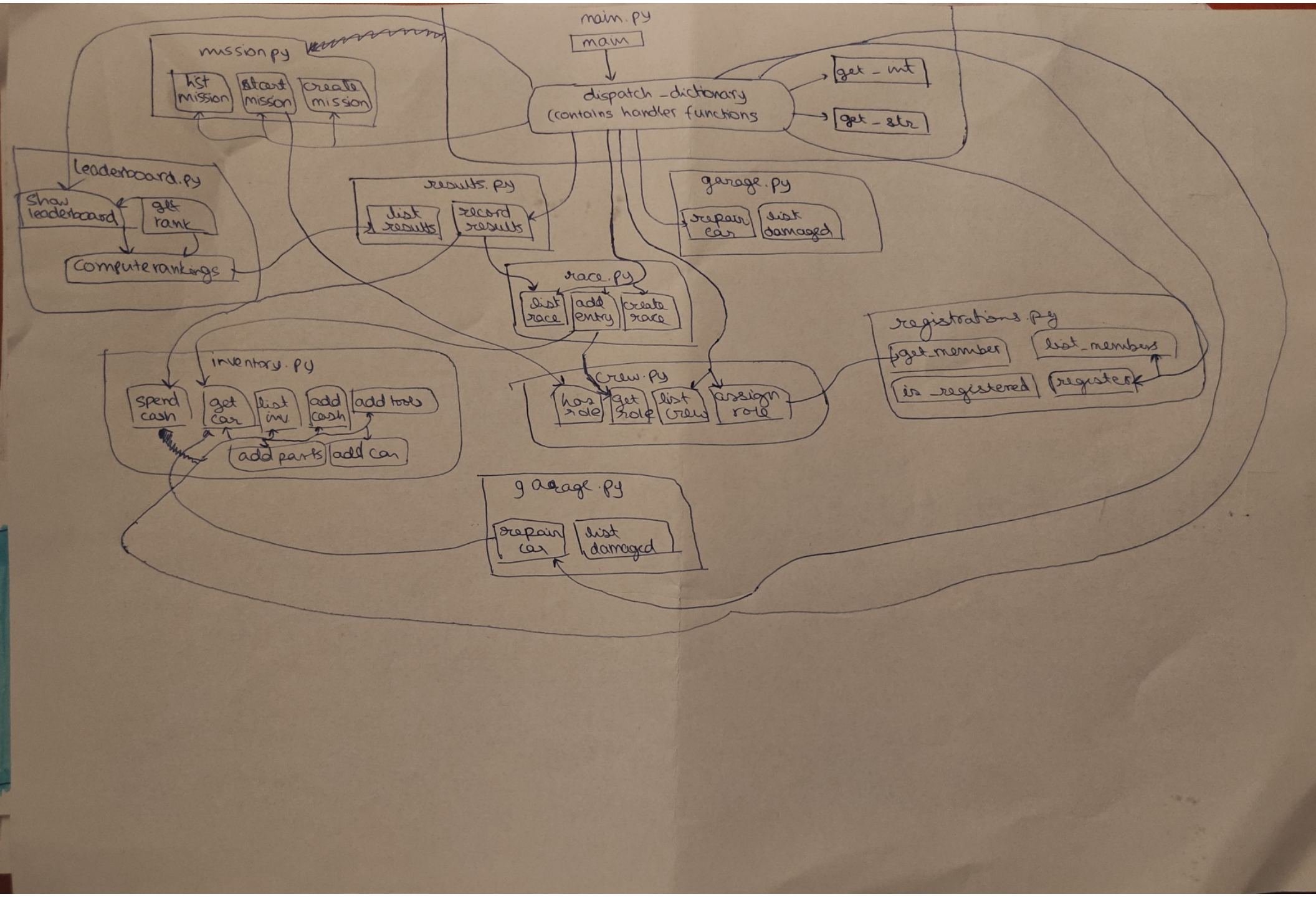
main.py
main
dispatch_dictionary
(contains handler functions
get_int
get_str
mission.py
list mission
start mission
create mission
leaderboard.py
show leaderboard
get rank
compute rankings
results.py
list results
record results
garage.py
repair car
list damaged
race.py
list race
add entry
create race
registrations.py
get_member
list_members
is_registered
register
inventory.py
spend cash
get car
list inv.
add cash
add tools
add parts
add car
crew.py
has role
get role
list crew
assign role
garage.py
repair car
list damaged

Integration Testing

Part 2.2

Integration Test Design (`test_integration.py`)

Test Cases **1. `test_register_then_assign_to_crew`**
Register "Alice" as driver, assign to crew. `crew.get_role("Alice")` returns `"driver"`.
Modules: Registration, Crew
Why: Most basic cross-module flow. If Registration and Crew wiring breaks, nothing else works.

2. `test_crew_assign_unregistered_raises`
Assign unregistered to crew. Should raise `ValueError`.
Modules: Registration, Crew
Why: Unregistered members must not enter crew.

3. `test_re_registration_does_not_update_crew`
Originally, register Alice as driver, assign to crew, re-register as mechanic. `crew.get_role("Alice")` should return `"mechanic"`.
However, this was later changed so that registrations can't change roles which makes more sense.
Modules: Registration, Crew
Why: You should not be able to re-register.

4. `test_only_drivers_can_enter_race`
Register member as mechanic, assign to crew, try to add to race. Raises `ValueError`.
Modules: Registration, Crew, Race
Why: Validate `Race.add_entry` checking `crew.get_role(driver) != "driver"`.

5. `test_car_damaged_by_driver_a_blocks_driver_b`
Driver A races and damages a car. Driver B tries to enter a new race with that car. Raises `ValueError`.
Modules: Results, Inventory, Race
Why: Inventory is shared. Results marking a car `"damaged"` must block Race from accepting it.

6. `test_two_drivers_same_car_same_race`
Two drivers enter the same race with the same car. Both entries succeed.
Modules: Race, Crew, Inventory
Why: Documents that the code checks for duplicate drivers but not duplicate cars.

7. `test_full_race_lifecycle`
Register driver, assign crew, add car and cash, create race, add entry, record result. Verify prize deducted and result stored.

Modules: Registration, Crew, Inventory, Race, Results
Why: End-to-end test. If any single module's is wrong, this should fail.

8. `test_multiple_drivers_leaderboard_order_and_rank`
Two drivers finish with different totals. `show_leaderboard()` lists higher earner first. `get_rank` returns 1 and 2.
Modules: Registration, Crew, Inventory, Race, Results, Leaderboard
Why: Tests both `show_leaderboard()` sorting and `get_rank()` indexing with real pipeline data.

9. `test_leaderboard_accumulates_across_races`
One driver wins two races. Leaderboard shows the sum of both prizes.
Modules: Race, Results, Leaderboard
Why: Confirms `totals.get(driver, 0) + result["prize"]` accumulation works with real Results data.

10. `test_repair_consumes_parts_from_shared_inventory`
Add parts to inventory, repair a car, verify parts are gone.
Modules: Garage, Inventory
Why: Confirms `inventory.parts.remove(part)` removes parts.

11. `test_mission_starts_after_crew_assigned`
Create mission requiring `"driver"`, assign a driver to crew, start mission.
Modules: Registration, Crew, Mission
Why: `Mission.start_mission` calls `crew.has_role("driver")`. Role must be visible to Mission.

12. `test_mission_fails_without_required_role`
Create mission requiring `"mechanic"`, only a driver in crew. `start_mission` raises `ValueError`.
Modules: Crew, Mission
Why: Mission must not start if the crew cannot fulfill all required roles.

13. `test_leaderboard_reflects_live_results`
Record one result, check leaderboard. Then record another for the same driver; Total should update.
Modules: Results, Leaderboard, Crew, Registration, Inventory
Why: Proves leaderboard recomputes live from `results.list_results()` every call with no caching.

14. `test_leaderboard_zero_prize_result`
Record result with `prize=0`. Driver still appears on leaderboard with total 0.
Modules: Results, Leaderboard, Registration, Crew, Inventory
Why: A driver who raced but won nothing should still show up ranked last.

15. `test_leaderboard_empty_before_any_results`
No results recorded. `show_leaderboard()` returns `[]`.
Modules: Results, Leaderboard
Why: Leaderboard must not crash or return garbage with no data.

16. `test_garage_lists_car_as_damaged_after_result`
Record result with `car_damaged=True`. `garage.list_damaged()` includes that car.
Modules: Results, Inventory, Garage
Why: Results and Garage both access the same inventory state. Confirms they are consistent.

17. `test_insufficient_cash_blocks_result`
$50 in inventory, try to record $100 prize. Raises `ValueError`. Cash stays at $50.
Modules: Inventory, Results
Why: Confirms the cash guard works and a failed result does not partially deduct money.

18. `test_damage_repair_rerace_cycle`
Register driver, add car, cash, and parts, race with car damaged, repair, re-race. Leaderboard shows sum of both prizes.
Modules: Registration, Crew, Inventory, Race, Results, Garage, Leaderboard
Why: Touches every module and fully verifies that everything works.

Errors & Fixes I failed test cases 3 and 6:

- Test case 3 was fixed by re-evaluating the test case. You shouldn't be able to re-register.
- Test case 6 was fixed by checking if the car is already in an entry